ClarifySQL

Teaching a small language model when to ask before it writes SQL

Mid-submission report

Language Models and Agents

GUILD4AGENTS

2025201007 - Subhash Dangeti,2025202004 - Madha Manish,2025201002-Mannem Tejaswini,2025201020 - Sathvik Reddy

github.com/Sathviktr/LMA_ClarifySQL

1 What our project is about

Our project is about helping a language model decide when it should ask a question before writing SQL. For example, if someone asks, "Which is the best department?", the word "best" could mean highest marks, best placement rate or highest pass percentage. A model can write SQL that runs correctly but still answer a different question from the one the user meant.

We are building an agent that can look up database information, ask the user for a missing detail, write SQL when the request is clear, or explain that the database does not have the required information. We use Qwen3-4B-Instruct-2507, an open model, and train a small set of extra weights called an *adapter*. The main model weights stay fixed.

We want to answer two questions:

1. Does training the agent to ask for clarification work better than giving the same model instructions and examples in a prompt?
2. Does it help if the model writes down the missing details before asking its question? We call each missing detail a *slot*, such as the minimum age or the measure used for ranking.

At this stage, the agent, datasets, baseline experiments and main training runs are ready. We have results on the validation set. The final test comparison, human evaluation, demo and final report are still pending.

2 Progress so far

Part of the project	Current progress
Setup and GPU checks	Completed the model-loading and training pilots on Kaggle T4 GPUs.
Data preparation	Added Spider, CoSQL and TrustSQL loaders, fixed the splits, and built and reviewed the episode dataset.
Tools and SQL safety	Implemented schema lookup, sample values, glossary search and read-only SQL execution.
Agent and baselines	Implemented the agent loop, conversation memory, simulated user, direct SQL baselines and prompted agent. Validation runs are complete.
Fine-tuning	Completed a small pilot, compared learning rates and checkpoints, and trained the main model with two seeds and the no-slot model.
Final evaluation	Main test runs, the full-schema ablation, CoSQL scoring, statistical analysis and evaluation audits are pending.
Demo and final submission	The fictional university database demo and final submission package are pending.

2.1 How the agent works

The agent chooses one of four actions:

- **Inspect:** use a tool to check a table, look at example values, or search for a definition.
- **Clarify:** ask the user about something missing from the request.
- **Execute:** give the clarified request to the SQL writer and run the query.
- **Abstain:** explain which required information is not stored in the database.

The trained adapter is used for choosing these actions. The SQL writer uses the same base model with the adapter turned off. Keeping the writer fixed helps us compare the clarification policies fairly.

The controller stores the user's replies and checks the model's outputs. For each episode, it allows at most three tool inspections, two questions, two SQL repairs and eight model calls for the policy and writer. The simulated user is separate from this call budget. SQL runs through a read-only connection with checks against writes and multiple statements, along with limits on rows and running time.

We also ran the local test suite with the benchmark data configured. **All 178 tests passed.** These tests cover the tools, safety checks, data splits, agent behavior, evaluation code, notebook consistency and small-model training checks.

3 Datasets we are using

Dataset	How we use it
Spider 1.0	Provides database files, questions and correct SQL. We use it to make clear and unclear requests while keeping the correct SQL as the hidden answer. The source data includes 7,000 core training questions on 140 databases and 1,034 dev questions on 20 databases.
CoSQL	Provides real conversations about databases. We reviewed 60 dev turns with their earlier dialogue and kept 59 for a separate evaluation. These contain 45 execute, 13 clarify and 1 abstain labels. The agent has not yet been evaluated on this slice.
TrustSQL	Provides questions that the database cannot answer. We use reviewed examples in the unanswerable part of our test set.

3.1 Our episode dataset

An *episode* is one request on one database, together with the hidden intended meaning and the expected action. We have five types:

- **Clear:** the request has enough detail to write SQL directly.
- **Ambiguous:** one detail is missing, so the agent should ask.
- **Two-slot:** two details are missing. Answering only one should not make the agent assume the other.
- **Definition-resolved:** the missing detail is explained in a glossary. The agent should find it instead of asking the user.
- **Unanswerable:** the requested information is not stored, so the agent should explain this and stop.

Episode type	Training	Validation	Test
Clear	130	22	79
Ambiguous	104	13	47
Two-slot	10	2	8
Definition-resolved	21	5	14
Unanswerable	42	11	40
Total	307	53	188

The split file assigns 130 databases to training, 10 to validation and 20 to testing. No database is assigned to more than one split. Four validation episodes from an empty database were removed, so the retained validation episodes cover nine of the ten assigned databases. Our test set is built from Spider's public dev databases; it is not Spider's hidden official test set.

The training episodes were converted into **602 training states**. A state is the request and conversation so far, with the next correct action as the target. There are 101 validation states. Six training states were dropped because they exceeded the 1,024-token limit. Hidden SQL and future user replies are not included in the policy input.

3.2 Review and data quality

We used model paraphrasing only for training requests. The non-clear validation and test requests were written or edited by team members. Automatic checks look for broken SQL, hidden answers accidentally left in a request, and problems with glossary entries.

The completed annotation sheets record independent labels from two annotators for 57 validation and 196 test episodes before all removals. Their Cohen's kappa scores are 0.971 and 0.926. Kappa measures agreement between annotators while accounting for agreement by chance. Disagreements were resolved before the final data files were prepared.

The training audit covered 144 episodes and removed 33 weak examples, leaving 307 training episodes. Review reduced validation from 60 to 53 episodes and test from 200 to 188. We kept the smaller reviewed sets rather than adding weak examples just to reach the planned sizes. The dataset-review sheets are complete; the separate audits of generated questions and simulated-user replies are still pending.

We use public benchmark data and will use made-up records for the university demo. The data download manifest records Spider and CoSQL as CC BY-SA 4.0 and TrustSQL as CC BY 4.0. The derived dataset needs the required attribution and licence information when released.

4 Experiments completed

4.1 Baselines and ablations

A *baseline* gives us something to compare our trained agent against. An *ablation* removes or changes one part of the system to check whether that part helps.

System	What it does	Status
B1	Writes SQL directly, without asking questions or repairing errors.	Validation done
B2	Same as B1, but can repair an SQL error up to two times.	Validation done
B3 / A1	Uses the full agent with instructions and worked examples, without fine-tuning.	Validation done
B3-zero	Same prompted agent without worked examples.	Validation done
Our SFT agent	Uses the trained clarification adapter. We trained it with two random seeds.	Validation done
Oracle	Gives the original, fully specified question to the same SQL writer.	Validation done
A2: no slots	Trains the agent to ask directly, without listing slots or possible meanings.	Validation done
A3: full schema	Gives the full schema and all glossary definitions at the start.	Not run yet

B3 and our agent use the same tools, SQL writer, limits and output checks. If the policy returns an invalid JSON action, both get the same single format retry. This helps us check the effect of training rather than giving one system extra tools or attempts.

For repeatable experiments, we use a simulated user. The base model reads the agent's question and selects which hidden detail it asks about. A fixed template then replies with only that detail. It does not read the slots written by the agent. We still need to check these simulated replies by hand.

4.2 Training setup

We use QLoRA to train the adapter while keeping the base model fixed. The base weights are stored in 4-bit form to save memory, with FP16 computation. We use LoRA rank 16, alpha 32, dropout 0.05 and gradient checkpointing. There are about 33 million trainable parameters.

We first ran a pilot on 100 training states for two epochs. For the full runs, we used a batch of one state at a time and accumulated gradients over 16 states before updating the weights. Training ran for three epochs with a 1,024-token limit. The loss is calculated only on the target action tokens. A

random seed controls the adapter's starting weights and the training shuffle. Running two seeds helps check whether the improvement depends on one particular run. The main runs used about 6.53 GB of peak allocated GPU memory, and A2 used about 6.46 GB. The third epoch of the selected run processed about 296 training tokens per second.

We compared two learning rates and checked each saved epoch on full validation episodes:

Learning rate	Epoch	Execution accuracy (%)	Decision F1
2×10^{-4}	1	73.81	0.9437
2×10^{-4}	2	76.19	0.9453
2×10^{-4}	3	78.57	0.9827
1×10^{-4}	1	76.19	0.9250
1×10^{-4}	2	76.19	0.9830
1×10^{-4}	3	78.57	0.9830

Our selection rule uses decision F1 rounded to two decimals, followed by slot accuracy. Two candidates were still tied after this rule, so the code kept the first one in its candidate order. We selected **learning rate 2×10^{-4} at epoch 3**. We then used the same setting for seed 2 and A2. We will mention the tie in the final report instead of presenting this as a unique winning setting.

5 Results so far

These are validation results. We used validation to choose prompts and checkpoints, so these numbers are not our final test results. All systems below are scored on the same 53 retained episodes.

Execution accuracy checks whether the final query gives the correct result. It is measured on the 42 answerable episodes. Decision macro-F1 checks how well the model chooses clarify, execute or abstain across all 53 episodes. It gives each decision category equal importance. "Ask on clear" measures unnecessary questions on 22 clear requests. Correct abstention is measured on 11 unanswerable requests.

System	Execution (%)	Decision F1	Ask on clear (%)	Abstention (%)
B1: direct SQL	50.00	0.2250	0.00	0.00
B2: direct + repair	52.38	0.2250	0.00	0.00
B3: prompted agent	64.29	0.7497	27.27	100.00
Our agent, seed 1	78.57	0.9827	0.00	100.00
Our agent, seed 2	78.57	0.9443	4.55	100.00
A2: no slots	80.95	0.9607	0.00	90.91
Oracle	80.95	0.2250	0.00	0.00

The oracle always tries to execute, so it is useful for checking the SQL writer, not for comparing abstention decisions.

5.1 What we can say from these results

- **Fine-tuning helps on validation.** B3 answers 27 of 42 answerable episodes correctly, while both trained seeds answer 33 correctly. That is an increase of about 14.3 percentage points.
- **The trained agent asks fewer unnecessary questions.** B3 asks on 6 of 22 clear requests. Seed 1 asks on none, and seed 2 asks on one.

- **The trained agent uses definitions more successfully.** B3 solves 1 of 5 definition-resolved requests. Both trained seeds and A2 solve all 5.
- **The slot ablation gives mixed results.** A2 gets 34 of 42 executions correct, one more than the main agent, but correctly abstains on 10 of 11 unanswerable requests instead of all 11. We cannot yet say that listing slots improves the final answer.
- **Worked examples help the prompted baseline.** Without examples, B3-zero gets 42.86% execution accuracy and 0.7040 decision F1, compared with 64.29% and 0.7497 for B3. We therefore keep the worked examples in B3.

The current automatic slot scores are 40.00% for B3 and 73.33% for both trained seeds, measured on 15 ambiguous episodes. This score checks whether the agent names the missing details. It has wording-matching problems, so we will use human question review as well. Both seeds have the same execution accuracy, but their decision scores differ. Two seeds are a small robustness check and are not enough to measure training variation reliably.

B3, both trained seeds and A2 have no invalid policy outputs or budget-limit outcomes in the retained validation results. The 20-case safety fixture also passes. We still need a final model-driven safety demonstration and a clear count of attempted and executed violations.

5.2 Calls and computation

System	Policy/writer calls	Input tokens	Generation time (s)
B3	213	155,813	293.7
Our agent, seed 1	140	58,639	144.5
Our agent, seed 2	138	57,415	140.2
A2	140	54,722	146.0

These are the policy and SQL-writer costs recorded by the controller. They do not include simulated-user inference, so we will add that before reporting a total cost. Training and inference use free Kaggle/Colab GPU resources, with no paid model API calls.

5.3 Separate SQL-writer checks

We also tested the frozen SQL writer on the original Spider questions. These runs do not include clarification. On 537 validation questions, execution accuracy changes from 88.64% before repair to 89.01% after repair. On 1,034 Spider-dev questions, it changes from 76.40% to 77.08%.

The official test-suite evaluator gives 67.99% before repair and 68.47% after repair on the Spider-dev run. It checks the SQL on database variants, which can catch mistakes missed by checking only one database. This is separate from our pending 188-episode agent test. The public test databases have already been used for this writer check, and public benchmark data may also have appeared in model pretraining. We will state these limitations in the final report.

6 Challenges and how we are handling them

Challenge	What we have done / what we will do
Some generated requests sound unnatural or still contain the answer.	Added checks, used paraphrasing only for training, edited evaluation requests by hand and dropped weak examples during review.
Some CoSQL labels do not fit the request and earlier conversation.	Reviewed 60 turns with context and kept 59 cleaned examples. We will report this slice separately.
One validation database has no rows, which can make incorrect SQL appear correct.	Removed its four episodes and rescored the older baselines on the same remaining 53 episodes. The original traces are kept for reference.
Training has to fit on a free T4 GPU.	Used 4-bit weights, FP16, small batches and gradient checkpointing. We also removed an incompatible unused torchao package in the Kaggle notebook.
Lower validation loss does not always mean better agent decisions.	Chose the checkpoint using complete validation episodes rather than training loss alone, and recorded the tie in the selection rule.
The automatic slot score can reject a correct phrase just because it uses different words.	Existing checks identify wording issues in some reference slots, including eight test terms. We will review and record corrections before final runs and add blind human question ratings.
The current intent check only verifies that numbers from the user's reply appear in the SQL.	We will describe this limit clearly and inspect errors involving the wrong column, ranking measure or comparison. Any change before testing must apply equally to the compared systems.
The SQL writer and simulated user can still make mistakes.	Keep the writer fixed for comparison, use the oracle to help separate errors, and manually check 50 simulator decisions per system.
The final adapters are saved in Kaggle outputs rather than this Git clone.	Confirm access to all three adapters before the test runs and add clear artifact links or a manifest for the final submission.

7 Work left for the final submission

Our main remaining task is to check whether the validation improvements also hold on the final episode test set.

1. Review the reference-slot wording issues, record the changes and freeze the dataset, prompts, model checkpoints and limits.
2. Run B1, B2, B3, oracle, both trained seeds, A2 and A3 on the same 188 test episodes. There are 148 answerable and 40 unanswerable cases.
3. Run B3 and our agent on the 59 cleaned CoSQL turns, using the earlier conversation and scoring the first decision.
4. Extend test-suite scoring to the agent outputs and report results by request type. The clarified values must be predicted, not replaced with gold values.
5. Estimate uncertainty with a paired bootstrap over the 20 test databases. This repeatedly samples

whole databases and compares the two systems on the same sample. We use databases rather than individual questions because questions from one database are related. The main comparison is our agent versus B3 on execution accuracy and decision F1; other comparisons are exploratory.

6. Ask two team members to rate questions from B3, our agent and A2 on the same 40 ambiguous test episodes, without showing the system names. They will check whether a question targets the missing detail, fits the schema, avoids repetition and lets the user resolve the request. Also audit 50 simulated-user decisions per system.
7. Build the fictional university demo, finish error analysis and total cost reporting, and prepare the final report and reproducibility material.

The demo will show a clear request, a glossary lookup, a useful question, a partial reply, missing information, an SQL repair and a blocked write. Extra ablations without memory, intent checks or repair are optional; the required runs and human evaluation come first.

8 Plan for the next 15 days

Day	Date	Planned work
0	30 Oct	Submit the mid-report and verify that the GitHub code is accessible. Confirm access to the three trained adapters.
1–2	31 Oct–1 Nov	Review slot wording, freeze the evaluation settings, and prepare the final test notebook and CoSQL input. Check the runner on a small sample.
3–4	2–3 Nov	Run B1, B2, oracle, B3 and both trained seeds on the test episodes. Save outputs so interrupted runs can be resumed.
5	4 Nov	Run A2, A3 and the CoSQL evaluation. Check that every expected episode has an output.
6–7	5–6 Nov	Calculate execution and test-suite scores, confidence intervals, results by request type and computation costs.
8–9	7–8 Nov	Complete the two-person question review and simulator audit. Discuss disagreements and summarize the findings.
10–11	9–10 Nov	Build and check the university demo, and finish the safety examples.
12–13	11–12 Nov	Write the final report, prepare tables and error analysis, and document the environment, adapter artifacts and AI assistance.
14	13 Nov	Test from a clean checkout, check links and artifacts, and leave time for reruns or fixes.
15	14 Nov	Review and submit the final report, code, artifact links and demo.

The main risks are GPU availability, access to saved adapters and scheduling the human review. We will check the adapters first, save predictions during runs and keep Colab as a backup. If time becomes tight, we will drop optional experiments before reducing the required evaluation.

9 Code and supporting files

Our repository is at https://github.com/Sathviktr/LMA_ClarifySQL. It contains the implementation, data scripts, split files, derived episodes, annotation sheets, tests, notebooks, training records and saved experiment results. Raw databases, model caches and large adapter weights are kept outside Git.

The README phase labels are older than the actual implementation. This report uses the source code, tests and saved result files to describe progress. Before submitting the GitHub link, we need to check that the latest code is pushed and that the instructor can access it. The final submission also needs an accessible reference to the saved adapter weights.

To run the local checks, install the package and requirements, set `CLARIFYSQL_DATA` to the benchmark directory, and run `python -m pytest`. The annotation checker is `scripts/annotation_status.py`. Notebooks 01–06 cover the completed GPU stages.

The main evidence for the numbers in this report is available in these repository files:

- Prompted baseline results, trained seed 1, trained seed 2 and no-slot ablation.
- Checkpoint selection, training-state statistics and training audit.
- Database splits, annotation details and separate SQL-writer results.

AI assistance: Claude Code was used for much of the implementation. Codex helped review the repository and prepare this report from the existing results. The team needs to verify the claims, understand the submitted code and include any additional AI-use details required by the course.